vae model(no use now)

In [10]:

```
import os
import numpy as np
import tensorflow as tf
from tensorflow.keras import layers
from tensorflow.keras.layers import Input, Dense, Conv2D, Flatten, Reshape, Conv2DTransp
ose
from tensorflow.keras.models import Model, load_model
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.losses import mse
from sklearn.model_selection import train_test_split
from tensorflow.keras.preprocessing.image import load_img, img_to_array
import matplotlib.pyplot as plt

# Set image size and path to dataset
image_size = (64, 64)
image_channels = 3
dataset_path = r"E:\dl_miniproject\Pokemon_Dataset\pokemon_augmented"

# Function to load and preprocess images
def load_images_from_folder(folder, image_size):
    images = []
    for filename in os.listdir(folder):
        if filename.endswith(".jpg"):
            img_path = os.path.join(folder, filename)
            img = load_img(img_path, target_size=image_size)
            img_array = img_to_array(img) / 255.0
            images.append(img_array)
    return np.array(images)

# Load images from the dataset folder
images = load_images_from_folder(dataset_path, image_size)

train_images, val_images = train_test_split(images, test_size=0.2, random_state=42)

if train_images.ndim == 3:
    train_images = np.expand_dims(train_images, axis=-1)

if val_images.ndim == 3:
    val_images = np.expand_dims(val_images, axis=-1)

assert not np.any(np.isnan(train_images)), "NaN values found in training images"
assert not np.any(np.isnan(val_images)), "NaN values found in validation images"

print(f"Train data range: {train_images.min()} - {train_images.max()}")
print(f"Validation data range: {val_images.min()} - {val_images.max()}")

print("Train images shape:", train_images.shape)  # Should be (batch_size, height, width,
channels)
print("Validation images shape:", val_images.shape)  # Should be (batch_size, height, wid
th, channels)

# Define the VAE architecture
latent_dim = 50

# Encoder
input_img = Input(shape=(image_size[0], image_size[1], image_channels))

x = Conv2D(32, (3, 3), activation='relu', padding='same', kernel_initializer='glorot_uni
form')(input_img)
x = Conv2D(64, (3, 3), activation='relu', padding='same', kernel_initializer='glorot_uni
form')(x)

x = Flatten()(x)
x = Dense(128, activation='relu', kernel_initializer='he_normal')(x)
```

```
z_mean = Dense(latent_dim)(x)
z_log_var = Dense(latent_dim)(x)

# Define the SamplingLayer to replace the Lambda function
class SamplingLayer(layers.Layer):
    def call(self, inputs):
        z_mean, z_log_var = inputs
        batch = tf.shape(z_mean)[0]
        dim = tf.shape(z_mean)[1]
        epsilon = tf.keras.backend.random_normal(shape=(batch, dim))
        return z_mean + tf.exp(0.5 * z_log_var) * epsilon

# Use SamplingLayer instead of Lambda
z = SamplingLayer()([z_mean, z_log_var])

# Decoder
decoder_input = Input(shape=(latent_dim,))
x = Dense(128, activation='relu')(decoder_input)
x = Dense(image_size[0] * image_size[1] * image_channels, activation='sigmoid')(x)
x = Reshape((image_size[0], image_size[1], image_channels))(x)
x = Conv2DTranspose(64, (3, 3), activation='relu', padding='same')(x)
x = Conv2DTranspose(32, (3, 3), activation='relu', padding='same')(x)
decoded_img = Conv2DTranspose(image_channels, (3, 3), activation='sigmoid', padding='sam
e')(x)

# Models
encoder = Model(input_img, [z_mean, z_log_var, z], name="encoder")
decoder = Model(decoder_input, decoded_img, name="decoder")

# Custom VAE loss function (reconstruction loss only)
def vae_loss(input_img, vae_output):
    reconstruction_loss = tf.reduce_mean(tf.keras.losses.binary_crossentropy(
        tf.keras.backend.flatten(input_img),
        tf.keras.backend.flatten(vae_output)
    ))
    return reconstruction_loss

# Define the VAE model
vae_output = decoder(z)
vae = Model(input_img, vae_output, name="vae")

# Compile the model with custom reconstruction loss (KL loss is handled inside SamplingLa
yer)
vae.compile(optimizer=tf.keras.optimizers.RMSprop(learning_rate=1e-4), loss=vae_loss)

# Model summary
vae.summary()

# Callbacks to handle NaN and debug losses
class TerminateOnNaN(tf.keras.callbacks.Callback):
    def on_epoch_end(self, epoch, logs=None):
        if logs is not None:
            if np.isnan(logs.get('loss')) or np.isnan(logs.get('val_loss')):
                print(f"NaN detected in epoch {epoch}. Stopping training.")
                self.model.stop_training = True

class DebugLosses(tf.keras.callbacks.Callback):
    def on_epoch_end(self, epoch, logs=None):
        if logs is not None:
            reconstruction_loss = logs.get('loss')
            val_reconstruction_loss = logs.get('val_loss')
            print(f"Epoch {epoch}: Reconstruction loss = {reconstruction_loss}, Validati
on loss = {val_reconstruction_loss}")

# Training the model
history = vae.fit(train_images, train_images,
                  epochs=10,
                  batch_size=32,
                  validation_data=(val_images, val_images),
                  callbacks=[TerminateOnNaN(), DebugLosses()])
```

```python
# Save the entire VAE model and weights separately
vae.save(r"E:\dl_miniproject\vae_model_1.h5")
vae.save_weights(r"E:\dl_miniproject\vae_weights_1.weights.h5")

# Save the encoder and decoder models separately
encoder.save(r"E:\dl_miniproject\vae_encoder_model.h5")
decoder.save(r"E:\dl_miniproject\vae_decoder_model.h5")

# Plot training and validation loss
plt.plot(history.history['loss'], label='train_loss')
plt.plot(history.history['val_loss'], label='val_loss')
plt.title('Model Loss')
plt.ylabel('Loss')
plt.xlabel('Epoch')
plt.legend(loc='upper right')
plt.show()
```

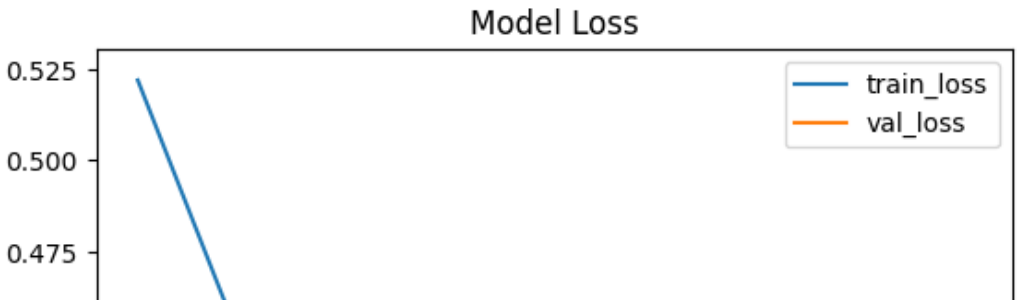
```
Train data range: 0.0 - 1.0
Validation data range: 0.0 - 1.0
Train images shape: (3592, 64, 64, 3)
Validation images shape: (898, 64, 64, 3)
Model: "vae"
__________________________________________________________________________________________________
 Layer (type)                    Output Shape         Param #     Connected to
==================================================================================================
 input_3 (InputLayer)            [(None, 64, 64, 3)]  0           []

 conv2d_2 (Conv2D)               (None, 64, 64, 32)   896         ['input_3[0][0]']

 conv2d_3 (Conv2D)               (None, 64, 64, 64)   18496       ['conv2d_2[0][0]']

 flatten_1 (Flatten)             (None, 262144)       0           ['conv2d_3[0][0]']

 dense_5 (Dense)                 (None, 128)          33554560    ['flatten_1[0][0]']

 dense_6 (Dense)                 (None, 50)           6450        ['dense_5[0][0]']

 dense_7 (Dense)                 (None, 50)           6450        ['dense_5[0][0]']

 sampling_layer_1 (SamplingLaye  (None, 50)           0           ['dense_6[0][0]',
 r)                                                                 'dense_7[0][0]']

 decoder (Functional)            (None, 64, 64, 3)    1612803     ['sampling_layer_1[0][0]
']

==================================================================================================
Total params: 35,199,655
Trainable params: 35,199,655
Non-trainable params: 0
__________________________________________________________________________________________________
```

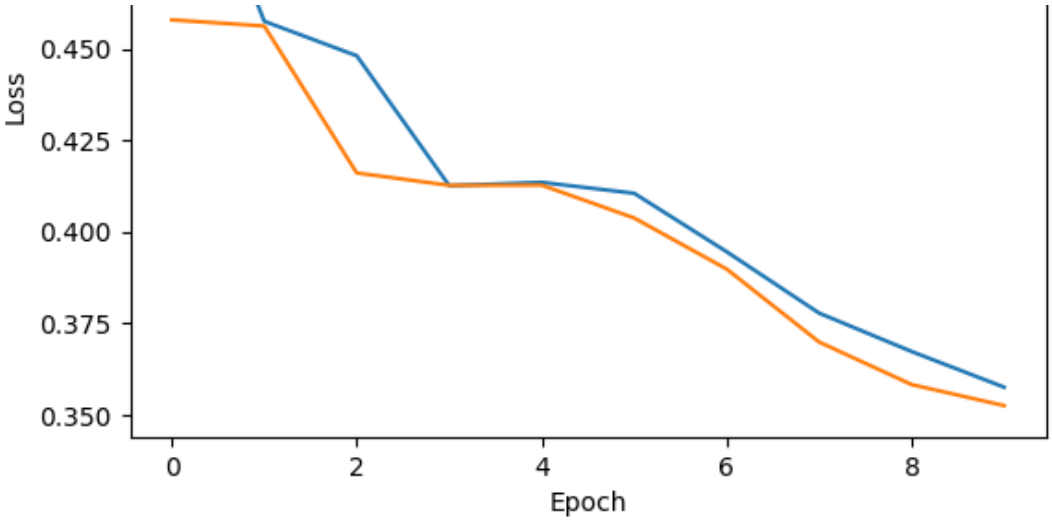
```
_________
Epoch 1/10
113/113 [==============================] - ETA: 0s - loss: 0.5219Epoch 0: Reconstruction
loss = 0.521946907043457, Validation loss = 0.45788493752479553
113/113 [==============================] - 89s 778ms/step - loss: 0.5219 - val_loss: 0.45
79
Epoch 2/10
113/113 [==============================] - ETA: 0s - loss: 0.4575Epoch 1: Reconstruction
loss = 0.4575003683567047, Validation loss = 0.45620596408843994
113/113 [==============================] - 87s 768ms/step - loss: 0.4575 - val_loss: 0.45
62
Epoch 3/10
113/113 [==============================] - ETA: 0s - loss: 0.4481Epoch 2: Reconstruction
loss = 0.4480730891227722, Validation loss = 0.41606807708740234
113/113 [==============================] - 87s 770ms/step - loss: 0.4481 - val_loss: 0.41
61
Epoch 4/10
113/113 [==============================] - ETA: 0s - loss: 0.4126Epoch 3: Reconstruction
loss = 0.41260161995887756, Validation loss = 0.4126887321472168
113/113 [==============================] - 88s 780ms/step - loss: 0.4126 - val_loss: 0.41
27
Epoch 5/10
113/113 [==============================] - ETA: 0s - loss: 0.4135Epoch 4: Reconstruction
loss = 0.41351020336151123, Validation loss = 0.4127486050128937
113/113 [==============================] - 90s 794ms/step - loss: 0.4135 - val_loss: 0.41
27
Epoch 6/10
113/113 [==============================] - ETA: 0s - loss: 0.4105Epoch 5: Reconstruction
loss = 0.4105154573917389, Validation loss = 0.4037725627422333
113/113 [==============================] - 88s 781ms/step - loss: 0.4105 - val_loss: 0.40
38
Epoch 7/10
113/113 [==============================] - ETA: 0s - loss: 0.3946Epoch 6: Reconstruction
loss = 0.3945563733577728, Validation loss = 0.38982120156288147
113/113 [==============================] - 86s 763ms/step - loss: 0.3946 - val_loss: 0.38
98
Epoch 8/10
113/113 [==============================] - ETA: 0s - loss: 0.3778Epoch 7: Reconstruction
loss = 0.3777863383293152, Validation loss = 0.36989349126815796
113/113 [==============================] - 87s 773ms/step - loss: 0.3778 - val_loss: 0.36
99
Epoch 9/10
113/113 [==============================] - ETA: 0s - loss: 0.3673Epoch 8: Reconstruction
loss = 0.36729809641838074, Validation loss = 0.35827258229255676
113/113 [==============================] - 85s 756ms/step - loss: 0.3673 - val_loss: 0.35
83
Epoch 10/10
113/113 [==============================] - ETA: 0s - loss: 0.3576Epoch 9: Reconstruction
loss = 0.3575553596019745, Validation loss = 0.3525252640247345
113/113 [==============================] - 83s 737ms/step - loss: 0.3576 - val_loss: 0.35
25
WARNING:tensorflow:Compiled the loaded model, but the compiled metrics have yet to be bui
lt. `model.compile_metrics` will be empty until you train or evaluate the model.
```


```
WARNING:tensorflow:Compiled the loaded model, but the compiled metrics have yet to be bui
lt. `model.compile_metrics` will be empty until you train or evaluate the model.
```

```
WARNING:tensorflow:Compiled the loaded model, but the compiled metrics have yet to be bui
lt. `model.compile_metrics` will be empty until you train or evaluate the model.
```

```
WARNING:tensorflow:Compiled the loaded model, but the compiled metrics have yet to be bui
lt. `model.compile_metrics` will be empty until you train or evaluate the model.
```



baground removing when needed

In [ ]:

```
from rembg import remove
from PIL import Image
import numpy as np
import cv2

def remove_background_rembg(image_path, output_path):
    input_image = Image.open(image_path).convert("RGBA")
    output_image = remove(input_image)
    output_image = output_image.convert("RGB")
    output_image.save(output_path)

# Example usage:
human_image_path = r"E:\dl_miniproject\kunal.jpg"
output_path = r"E:\dl_miniproject\backgroundless_kunal.jpg"
remove_background_rembg(human_image_path, output_path)
```

feature extraction using vgg16

In [ ]:

```
import os
import numpy as np
import cv2
from tensorflow.keras.applications.vgg16 import VGG16, preprocess_input
from tensorflow.keras.preprocessing.image import img_to_array, load_img
from tensorflow.keras.models import Model
from sklearn.metrics.pairwise import cosine_similarity

base_model = VGG16(weights='imagenet')
model = Model(inputs=base_model.input, outputs=base_model.get_layer('fc1').output)

def preprocess_image(image_path):
    img = load_img(image_path, target_size=(224, 224))
    img = img_to_array(img)
    img = np.expand_dims(img, axis=0)
    img = preprocess_input(img)
    return img

def extract_features(image_path, model):
    img = preprocess_image(image_path)
    features = model.predict(img)
    return features.flatten()

def load_dataset(folder_path, model):
    features = []
    filenames = []
    for subdir, _, files in os.walk(folder_path):
        for file in files:
```

```
            file_path = os.path.join(subdir, file)
            feature = extract_features(file_path, model)
            features.append(feature)
            filenames.append(file_path)
    return np.array(features), filenames

#datasets
#human_dataset_path = "E:\dl_miniproject\gender_dataset\humans_mixed"
pokemon_dataset_path = "E:\dl_miniproject\Pokemon_Dataset\Pokemon Dataset copy"

#features
human_features, human_filenames = load_dataset(human_dataset_path, model)
pokemon_features, pokemon_filenames = load_dataset(pokemon_dataset_path, model)

#feature saveing
human_features_path = "E:\\dl_miniproject\\human_features_3.npy"
human_filenames_path = "E:\\dl_miniproject\\human_filenames_3.npy"
pokemon_features_path = "E:\\dl_miniproject\\pokemon_new_features_3.npy"
pokemon_filenames_path = "E:\\dl_miniproject\\pokemon_new_filenames_3.npy"

"""
# Check if Pokémon features file exists, if not create it
if not os.path.exists(pokemon_features_path):
    pokemon_features = [extract_features(img_path, model) for img_path in pokemon_filenam
es]
    pokemon_features = np.array(pokemon_features)
    np.save(pokemon_features_path, pokemon_features)
else:
    pokemon_features = np.load(pokemon_features_path)

# Check if Pokémon filenames file exists, if not create it
if not os.path.exists(pokemon_filenames_path):
    # Assuming you have a way to generate or list Pokémon filenames
    pokemon_filenames = ["path_to_pokemon_image1", "path_to_pokemon_image2"]  # Example p
aths
    np.save(pokemon_filenames_path, pokemon_filenames)
else:
    pokemon_filenames = np.load(pokemon_filenames_path)"""
```

loading extracted features and trying cosine similarity on thjem

In [2]:

```
import numpy as np
from sklearn.metrics.pairwise import cosine_similarity

# Load the features and filenames
human_features = np.load(r"E:\dl_miniproject\human_features_3.npy")
human_filenames = np.load(r"E:\dl_miniproject\human_filenames_3.npy")
pokemon_features = np.load(r"E:\dl_miniproject\pokemon_features_3.npy")
pokemon_filenames = np.load(r"E:\dl_miniproject\pokemon_filenames_3.npy")

# Initialize a list to store the most similar Pokémon for each human
most_similar_pokemon = []

# Calculate cosine similarity for each human image
for human_feature in human_features:
    # Compute the cosine similarity between the current human image and all Pokémon image
s
    similarities = cosine_similarity([human_feature], pokemon_features)

    # Find the index of the most similar Pokémon
    most_similar_idx = np.argmax(similarities)

    # Store the corresponding Pokémon filename
    most_similar_pokemon.append(pokemon_filenames[most_similar_idx])

# Save the results to a file
np.save(r"E:\dl_miniproject\most_similar_pokemon.npy", most_similar_pokemon)

# Optionally, print a few sample matches
```

```
print("Sample human to Pokémon matches:")
for i in range(5):
    print(f"Human Image: {human_filenames[i]} --> Most Similar Pokémon: {most_similar_po
kemon[i]}")
```

```
Sample human to Pokémon matches:
Human Image: 1 (2)_0_1180.jpg --> Most Similar Pokémon: darkrai_0_2367.jpg
Human Image: 1 (2)_0_292.jpg --> Most Similar Pokémon: boldore_0_2176.jpg
Human Image: 1 (2)_0_3422.jpg --> Most Similar Pokémon: crustle_0_4377.jpg
Human Image: 1 (2)_0_4377.jpg --> Most Similar Pokémon: registeel_0_3289.jpg
Human Image: 1 (2)_0_7329.jpg --> Most Similar Pokémon: boldore_0_4996.jpg
```

generator and discriminator(old one. not used)

In [20]:

```
from tensorflow.keras.layers import LeakyReLU, BatchNormalization

# Generator
def build_generator(latent_dim):
    model = tf.keras.Sequential()
    model.add(Dense(128 * 8 * 8, activation="relu", input_dim=latent_dim))
    model.add(Reshape((8, 8, 128)))
    model.add(Conv2DTranspose(128, 3, strides=2, padding="same"))
    model.add(BatchNormalization())
    model.add(LeakyReLU(alpha=0.2))
    model.add(Conv2DTranspose(64, 3, strides=2, padding="same"))
    model.add(BatchNormalization())
    model.add(LeakyReLU(alpha=0.2))
    model.add(Conv2DTranspose(32, 3, strides=2, padding="same"))
    model.add(BatchNormalization())
    model.add(LeakyReLU(alpha=0.2))
    model.add(Conv2D(3, 3, activation="tanh", padding="same"))
    return model

# Discriminator
def build_discriminator(input_shape):
    model = tf.keras.Sequential()
    model.add(Conv2D(64, 3, strides=2, padding="same", input_shape=input_shape))
    model.add(LeakyReLU(alpha=0.2))
    model.add(Conv2D(128, 3, strides=2, padding="same"))
    model.add(LeakyReLU(alpha=0.2))
    model.add(Flatten())
    model.add(Dense(1, activation="sigmoid"))
    return model

# GAN loss function and optimizers
discriminator = build_discriminator(input_shape)
discriminator.compile(loss='binary_crossentropy', optimizer='adam')

generator = build_generator(latent_dim)
gan_input = Input(shape=(latent_dim,))
generated_img = generator(gan_input)

discriminator.trainable = False
gan_output = discriminator(generated_img)
gan = Model(gan_input, gan_output)
gan.compile(loss='binary_crossentropy', optimizer='adam')

# Train the GAN separately after training VAE
gan.summary()
```

```
Model: "model"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 input_8 (InputLayer)        [(None, 100)]             0

 sequential_1 (Sequential)   (None, 64, 64, 3)         1068995

 sequential (Sequential)     (None, 1)                 108417
```

```
=================================================================
Total params: 1,177,412
Trainable params: 1,068,547
Non-trainable params: 108,865
_________________________________________________________________
```

loading files(features, images..)

In [2]:

```
import numpy as np
import os
from tensorflow.keras.preprocessing.image import img_to_array, load_img
from tensorflow.keras.utils import to_categorical

# Load human features and filenames
human_features = np.load(r"E:\dl_miniproject\human_features_3.npy")  # Shape: (num_human
s, feature_dim)
human_filenames = np.load(r"E:\dl_miniproject\human_filenames_3.npy")

# Load Pokémon features and filenames
pokemon_features = np.load(r"E:\dl_miniproject\pokemon_features_3.npy")
pokemon_filenames = np.load(r"E:\dl_miniproject\pokemon_filenames_3.npy")

# Load the most similar Pokémon filenames for each human
most_similar_pokemon = np.load(r"E:\dl_miniproject\most_similar_pokemon.npy")

# Define the image size (e.g., 64x64)
image_size = (64, 64)
image_channels = 3  # RGB images

# Function to load images given filenames
def load_images(filenames, image_dir):
    images = []
    for filename in filenames:
        img_path = os.path.join(image_dir, filename)
        img = load_img(img_path, target_size=image_size)
        img_array = img_to_array(img) / 127.5 - 1  # Normalize to [-1, 1]
        images.append(img_array)
    return np.array(images)

# Directory containing Pokémon images
pokemon_image_dir = r"E:\dl_miniproject\Pokemon_Dataset\pokemon_augmented"

# Load Pokémon images corresponding to the most similar ones
pokemon_images = load_images(most_similar_pokemon, pokemon_image_dir)


# Ensure that human_features and pokemon_images have the same number of samples
assert human_features.shape[0] == pokemon_images.shape[0], "Mismatch in number of samples
."

# Define the latent dimension for noise vector
latent_dim = 100  # You can adjust this

# The condition vector dimension is the same as human_features' second dimension
condition_dim = human_features.shape[1]

# Generator input dimension is noise + condition
generator_input_dim = latent_dim + condition_dim
```

generator

In [3]:

```
from tensorflow.keras.layers import Input, Dense, Reshape, Conv2DTranspose, BatchNormaliz
ation, Activation, Concatenate
from tensorflow.keras.models import Model
```

```python
def build_generator(generator_input_dim, image_shape):
    # Generator input
    gen_input = Input(shape=(generator_input_dim,))

    # Split input into noise and condition
    noise = gen_input[:, :latent_dim]
    condition = gen_input[:, latent_dim:]

    # Optionally process condition vector
    x = Concatenate()([noise, condition])

    # Build generator layers
    x = Dense(8 * 8 * 256, activation='relu')(x)
    x = Reshape((8, 8, 256))(x)
    x = BatchNormalization()(x)

    x = Conv2DTranspose(128, kernel_size=4, strides=2, padding='same')(x)
    x = Activation('relu')(x)
    x = BatchNormalization()(x)

    x = Conv2DTranspose(64, kernel_size=4, strides=2, padding='same')(x)
    x = Activation('relu')(x)
    x = BatchNormalization()(x)

    x = Conv2DTranspose(image_channels, kernel_size=4, strides=2, padding='same')(x)
    x = Activation('tanh')(x)

    generator = Model(gen_input, x, name='generator')
    return generator

generator = build_generator(generator_input_dim, image_size + (image_channels,))
generator.summary()
```

```
Model: "generator"
__________________________________________________________________________________________________
 Layer (type)                    Output Shape         Param #     Connected to
==================================================================================================
 input_1 (InputLayer)            [(None, 25188)]      0           []


 tf.__operators__.getitem (Slic  (None, 100)          0           ['input_1[0][0]']
 ingOpLambda)


 tf.__operators__.getitem_1 (Sl  (None, 25088)        0           ['input_1[0][0]']
 icingOpLambda)


 concatenate (Concatenate)       (None, 25188)        0           ['tf.__operators__.geti
tem[0][0]'
                                                                  , 'tf.__operators__.ge
titem_1[0][
                                                                  0]']


 dense (Dense)                   (None, 16384)        412696576   ['concatenate[0][0]']


 reshape (Reshape)               (None, 8, 8, 256)    0           ['dense[0][0]']
```

```
 batch_normalization (BatchNorm  (None, 8, 8, 256)   1024        ['reshape[0][0]']

 alization)



 conv2d_transpose (Conv2DTransp  (None, 16, 16, 128)  524416     ['batch_normalization[0]
[0]']
 ose)



 activation (Activation)        (None, 16, 16, 128)  0           ['conv2d_transpose[0][0]
']


 batch_normalization_1 (BatchNo  (None, 16, 16, 128)  512        ['activation[0][0]']

 rmalization)



 conv2d_transpose_1 (Conv2DTran  (None, 32, 32, 64)  131136      ['batch_normalization_1[
0][0]']
 spose)



 activation_1 (Activation)      (None, 32, 32, 64)   0           ['conv2d_transpose_1[0][
0]']


 batch_normalization_2 (BatchNo  (None, 32, 32, 64)  256         ['activation_1[0][0]']

 rmalization)



 conv2d_transpose_2 (Conv2DTran  (None, 64, 64, 3)   3075        ['batch_normalization_2[
0][0]']
 spose)



 activation_2 (Activation)      (None, 64, 64, 3)    0           ['conv2d_transpose_2[0][
0]']


==================================================================================================
Total params: 413,356,995
Trainable params: 413,356,099
Non-trainable params: 896
__________________________________________________________________________________________________
```

discriminator

In [4]:

```
from tensorflow.keras.layers import Conv2D, LeakyReLU, Flatten, Dense, Dropout

def build_discriminator(image_shape, condition_dim):
    img_input = Input(shape=image_shape)
    condition_input = Input(shape=(condition_dim,))

    # Process image
    x = Conv2D(64, kernel_size=4, strides=2, padding='same')(img_input)
    x = LeakyReLU(0.2)(x)

    x = Conv2D(128, kernel_size=4, strides=2, padding='same')(x)
```

```
    x = LeakyReLU(0.2)(x)

    x = Flatten()(x)

    # Concatenate image features and condition
    x = Concatenate()([x, condition_input])

    x = Dense(512)(x)
    x = LeakyReLU(0.2)(x)
    x = Dropout(0.4)(x)

    x = Dense(1, activation='sigmoid')(x)

    discriminator = Model([img_input, condition_input], x, name='discriminator')
    return discriminator

discriminator = build_discriminator(image_size + (image_channels,), condition_dim)
discriminator.summary()
```

```
Model: "discriminator"
__________________________________________________________________________________________________
 Layer (type)                    Output Shape         Param #     Connected to
==================================================================================================
 input_2 (InputLayer)            [(None, 64, 64, 3)]  0           []

 conv2d (Conv2D)                 (None, 32, 32, 64)   3136        ['input_2[0][0]']

 leaky_re_lu (LeakyReLU)         (None, 32, 32, 64)   0           ['conv2d[0][0]']

 conv2d_1 (Conv2D)               (None, 16, 16, 128)  131200      ['leaky_re_lu[0][0]']

 leaky_re_lu_1 (LeakyReLU)       (None, 16, 16, 128)  0           ['conv2d_1[0][0]']

 flatten (Flatten)               (None, 32768)        0           ['leaky_re_lu_1[0][0]']

 input_3 (InputLayer)            [(None, 25088)]      0           []

 concatenate_1 (Concatenate)     (None, 57856)        0           ['flatten[0][0]',
                                                                   'input_3[0][0]']

 dense_1 (Dense)                 (None, 512)          29622784    ['concatenate_1[0][0]']

 leaky_re_lu_2 (LeakyReLU)       (None, 512)          0           ['dense_1[0][0]']

 dropout (Dropout)               (None, 512)          0           ['leaky_re_lu_2[0][0]']
```

```
 dense_2 (Dense)                (None, 1)            513         ['dropout[0][0]']


==================================================================================================
=========
Total params: 29,757,633
Trainable params: 29,757,633
Non-trainable params: 0
__________________________________________________________________________________________________
_________
```

compiuling cgan

In [5]:

```
from tensorflow.keras.optimizers import Adam

# Compile the discriminator
discriminator.compile(loss='binary_crossentropy', optimizer=Adam(0.0002, 0.5), metrics=[
'accuracy'])

# Build and compile the GAN
# Freeze discriminator's weights during generator training
discriminator.trainable = False

# Generator input
gen_input = Input(shape=(generator_input_dim,))
# Generated image
gen_image = generator(gen_input)
# Get the condition vector
gen_condition = gen_input[:, latent_dim:]
# Discriminator output
validity = discriminator([gen_image, gen_condition])

# Combined model (generator and discriminator)
gan = Model(gen_input, validity)
gan.compile(loss='binary_crossentropy', optimizer=Adam(0.0002, 0.5))
```

training cgan

In [14]:

```
import tensorflow as tf
from tensorflow.keras.callbacks import ModelCheckpoint
import numpy as np

# Check if GPU is available and print the status
gpus = tf.config.list_physical_devices('GPU')
if gpus:
    print(f"Using GPU: {gpus[0].name}")
else:
    print("Using CPU")

# Training parameters
epochs = 10000
batch_size = 32
save_interval = 500
latent_dim = 100  # Assuming you have a latent dimension defined

# Define paths to save models (using epoch number)
def save_models(epoch, generator, discriminator, gan):
    generator.save(f'E:/dl_miniproject/generator_{epoch}', save_format='tf')  # Using Sa
vedModel format
    print(f"Generator model saved at epoch {epoch}")

    discriminator.save(f'E:/dl_miniproject/discriminator_{epoch}', save_format='tf')  #
Using SavedModel format
    print(f"Discriminator model saved at epoch {epoch}")
```

```
    gan.save(f'E:/dl_miniproject/gan_{epoch}', save_format='tf')  # Using SavedModel for
mat
    print(f"GAN model saved at epoch {epoch}")

# Labels for real and fake images
real = np.ones((batch_size, 1))
fake = np.zeros((batch_size, 1))

# Prepare condition vectors and images
condition_vectors = human_features  # Make sure this is correctly loaded
real_images = pokemon_images  # Ensure the Pokémon images are preprocessed

# Number of samples
num_samples = condition_vectors.shape[0]

# Training loop
for epoch in range(epochs):
    # ---------------------
    #  Train Discriminator
    # ---------------------

    # Select a random batch of real images and conditions
    idx = np.random.randint(0, num_samples, batch_size)
    imgs = real_images[idx]
    conditions = condition_vectors[idx]

    # Sample noise
    noise = np.random.normal(0, 1, (batch_size, latent_dim))
    # Generator input
    gen_input = np.concatenate([noise, conditions], axis=1)
    # Generate images
    gen_imgs = generator.predict(gen_input)

    # Train discriminator on real images
    d_loss_real = discriminator.train_on_batch([imgs, conditions], real)
    # Train discriminator on fake images
    d_loss_fake = discriminator.train_on_batch([gen_imgs, conditions], fake)
    # Total discriminator loss
    d_loss = 0.5 * np.add(d_loss_real, d_loss_fake)

    # ---------------------
    #  Train Generator
    # ---------------------

    # Sample noise
    noise = np.random.normal(0, 1, (batch_size, latent_dim))
    # Get random conditions
    idx = np.random.randint(0, num_samples, batch_size)
    conditions = condition_vectors[idx]
    # Generator input
    gen_input = np.concatenate([noise, conditions], axis=1)

    # Train generator
    g_loss = gan.train_on_batch(gen_input, real)

    # Print progress and save models at intervals
    if epoch % save_interval == 0 or epoch == epochs - 1:
        print(f"{epoch} [D loss: {d_loss[0]:.4f}, acc.: {100 * d_loss[1]:.2f}%] [G loss:
{g_loss:.4f}]")

        # Save models at save_interval
        save_models(epoch, generator, discriminator, gan)
```

```
1/1 [==============================] - 1s 565ms/step
0 [D loss: 2.3251, acc.: 73.44%] [G loss: 40.3430]
WARNING:tensorflow:Compiled the loaded model, but the compiled metrics have yet to be bui
lt. `model.compile_metrics` will be empty until you train or evaluate the model.
```

```
WARNING:tensorflow:Compiled the loaded model, but the compiled metrics have yet to be bui
lt. `model.compile_metrics` will be empty until you train or evaluate the model.
WARNING:absl:Found untraced functions such as _jit_compiled_convolution_op, _jit_compiled
_convolution_op, _jit_compiled_convolution_op while saving (showing 3 of 3). These functi
ons will not be directly callable after loading.
```

```
ons will not be directly callable after loading.

INFO:tensorflow:Assets written to: E:/dl_miniproject/generator_0\assets

INFO:tensorflow:Assets written to: E:/dl_miniproject/generator_0\assets

Generator model saved at epoch 0

WARNING:absl:Found untraced functions such as _jit_compiled_convolution_op, _jit_compiled
_convolution_op, _update_step_xla while saving (showing 3 of 3). These functions will not
be directly callable after loading.

INFO:tensorflow:Assets written to: E:/dl_miniproject/discriminator_0\assets

INFO:tensorflow:Assets written to: E:/dl_miniproject/discriminator_0\assets

Discriminator model saved at epoch 0

WARNING:absl:Found untraced functions such as _update_step_xla, _jit_compiled_convolution
_op, _jit_compiled_convolution_op, _jit_compiled_convolution_op, _jit_compiled_convolutio
n_op while saving (showing 5 of 7). These functions will not be directly callable after l
oading.

INFO:tensorflow:Assets written to: E:/dl_miniproject/gan_0\assets

INFO:tensorflow:Assets written to: E:/dl_miniproject/gan_0\assets

GAN model saved at epoch 0
1/1 [==============================] - 0s 390ms/step
1/1 [==============================] - 0s 375ms/step
1/1 [==============================] - 0s 437ms/step
1/1 [==============================] - 0s 375ms/step
1/1 [==============================] - 0s 359ms/step
1/1 [==============================] - 0s 368ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 250ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 218ms/step
1/1 [==============================] - 0s 315ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 265ms/step
1/1 [==============================] - 0s 218ms/step
1/1 [==============================] - 0s 281ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 232ms/step
1/1 [==============================] - 0s 281ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 281ms/step
1/1 [==============================] - 0s 250ms/step
1/1 [==============================] - 0s 250ms/step
1/1 [==============================] - 0s 223ms/step
1/1 [==============================] - 0s 250ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 232ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 217ms/step
1/1 [==============================] - 0s 216ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 266ms/step
1/1 [==============================] - 0s 234ms/step
```

```
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 266ms/step
1/1 [==============================] - 0s 266ms/step
1/1 [==============================] - 0s 281ms/step
1/1 [==============================] - 0s 226ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 274ms/step
1/1 [==============================] - 0s 327ms/step
1/1 [==============================] - 0s 281ms/step
1/1 [==============================] - 0s 265ms/step
1/1 [==============================] - 0s 221ms/step
1/1 [==============================] - 0s 270ms/step
1/1 [==============================] - 0s 268ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 297ms/step
1/1 [==============================] - 0s 250ms/step
1/1 [==============================] - 0s 214ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 224ms/step
1/1 [==============================] - 0s 217ms/step
1/1 [==============================] - 0s 220ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 217ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 266ms/step
1/1 [==============================] - 0s 266ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 294ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 203ms/step
1/1 [==============================] - 0s 203ms/step
1/1 [==============================] - 0s 312ms/step
1/1 [==============================] - 0s 297ms/step
1/1 [==============================] - 0s 266ms/step
1/1 [==============================] - 0s 203ms/step
1/1 [==============================] - 0s 203ms/step
1/1 [==============================] - 0s 222ms/step
1/1 [==============================] - 0s 281ms/step
1/1 [==============================] - 0s 266ms/step
1/1 [==============================] - 0s 203ms/step
1/1 [==============================] - 0s 203ms/step
1/1 [==============================] - 0s 199ms/step
1/1 [==============================] - 0s 293ms/step
1/1 [==============================] - 0s 285ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 203ms/step
1/1 [==============================] - 0s 312ms/step
1/1 [==============================] - 0s 203ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 203ms/step
1/1 [==============================] - 0s 203ms/step
1/1 [==============================] - 0s 218ms/step
1/1 [==============================] - 0s 266ms/step
1/1 [==============================] - 0s 281ms/step
1/1 [==============================] - 0s 266ms/step
1/1 [==============================] - 0s 203ms/step
1/1 [==============================] - 0s 250ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 219ms/step
```

```
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 282ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 275ms/step
1/1 [==============================] - 0s 296ms/step
1/1 [==============================] - 0s 289ms/step
1/1 [==============================] - 0s 282ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 198ms/step
1/1 [==============================] - 0s 265ms/step
1/1 [==============================] - 0s 211ms/step
1/1 [==============================] - 0s 249ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 204ms/step
1/1 [==============================] - 0s 266ms/step
1/1 [==============================] - 0s 299ms/step
1/1 [==============================] - 0s 232ms/step
1/1 [==============================] - 0s 281ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 202ms/step
1/1 [==============================] - 0s 250ms/step
1/1 [==============================] - 0s 254ms/step
1/1 [==============================] - 0s 203ms/step
1/1 [==============================] - 0s 203ms/step
1/1 [==============================] - 0s 203ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 203ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 203ms/step
1/1 [==============================] - 0s 203ms/step
1/1 [==============================] - 0s 281ms/step
1/1 [==============================] - 0s 203ms/step
1/1 [==============================] - 0s 312ms/step
1/1 [==============================] - 0s 298ms/step
1/1 [==============================] - 0s 297ms/step
1/1 [==============================] - 0s 250ms/step
1/1 [==============================] - 0s 297ms/step
1/1 [==============================] - 0s 281ms/step
1/1 [==============================] - 0s 297ms/step
1/1 [==============================] - 0s 203ms/step
1/1 [==============================] - 0s 203ms/step
1/1 [==============================] - 0s 281ms/step
1/1 [==============================] - 0s 277ms/step
1/1 [==============================] - 0s 233ms/step
1/1 [==============================] - 0s 278ms/step
1/1 [==============================] - 0s 266ms/step
1/1 [==============================] - 0s 281ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 295ms/step
1/1 [==============================] - 0s 216ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 203ms/step
1/1 [==============================] - 0s 266ms/step
1/1 [==============================] - 0s 250ms/step
1/1 [==============================] - 0s 250ms/step
1/1 [==============================] - 0s 266ms/step
1/1 [==============================] - 0s 266ms/step
1/1 [==============================] - 0s 281ms/step
1/1 [==============================] - 0s 266ms/step
1/1 [==============================] - 0s 250ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 297ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 281ms/step
1/1 [==============================] - 0s 250ms/step
1/1 [==============================] - 0s 266ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 266ms/step
1/1 [==============================] - 0s 219ms/step
```

```
1/1 [==============================] - 0s 281ms/step
1/1 [==============================] - 0s 203ms/step
1/1 [==============================] - 0s 266ms/step
1/1 [==============================] - 0s 215ms/step
1/1 [==============================] - 0s 295ms/step
1/1 [==============================] - 0s 233ms/step
1/1 [==============================] - 0s 250ms/step
1/1 [==============================] - 0s 208ms/step
1/1 [==============================] - 0s 210ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 266ms/step
1/1 [==============================] - 0s 297ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 281ms/step
1/1 [==============================] - 0s 297ms/step
1/1 [==============================] - 0s 251ms/step
1/1 [==============================] - 0s 235ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 222ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 216ms/step
1/1 [==============================] - 0s 235ms/step
1/1 [==============================] - 0s 217ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 232ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 217ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 217ms/step
1/1 [==============================] - 0s 261ms/step
1/1 [==============================] - 0s 281ms/step
1/1 [==============================] - 0s 266ms/step
1/1 [==============================] - 0s 283ms/step
1/1 [==============================] - 0s 280ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 235ms/step
1/1 [==============================] - 0s 281ms/step
1/1 [==============================] - 0s 203ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 203ms/step
1/1 [==============================] - 0s 298ms/step
1/1 [==============================] - 0s 281ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 250ms/step
1/1 [==============================] - 0s 203ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 281ms/step
1/1 [==============================] - 0s 295ms/step
1/1 [==============================] - 0s 250ms/step
```

```
1/1 [==============================] - 0s 250ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 281ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 320ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 218ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 297ms/step
1/1 [==============================] - 0s 250ms/step
1/1 [==============================] - 0s 297ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 203ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 286ms/step
1/1 [==============================] - 0s 291ms/step
1/1 [==============================] - 0s 266ms/step
1/1 [==============================] - 0s 262ms/step
1/1 [==============================] - 0s 281ms/step
1/1 [==============================] - 0s 245ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 218ms/step
1/1 [==============================] - 0s 232ms/step
1/1 [==============================] - 0s 216ms/step
1/1 [==============================] - 0s 216ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 231ms/step
1/1 [==============================] - 0s 281ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 203ms/step
1/1 [==============================] - 0s 250ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 203ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 250ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 248ms/step
1/1 [==============================] - 0s 221ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 281ms/step
1/1 [==============================] - 0s 281ms/step
1/1 [==============================] - 0s 297ms/step
1/1 [==============================] - 0s 217ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 297ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 281ms/step
1/1 [==============================] - 0s 281ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 250ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 266ms/step
```

```
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 281ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 266ms/step
1/1 [==============================] - 0s 251ms/step
1/1 [==============================] - 0s 281ms/step
1/1 [==============================] - 0s 220ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 297ms/step
1/1 [==============================] - 0s 266ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 312ms/step
1/1 [==============================] - 0s 222ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 266ms/step
1/1 [==============================] - 0s 281ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 312ms/step
1/1 [==============================] - 0s 206ms/step
1/1 [==============================] - 0s 281ms/step
1/1 [==============================] - 0s 268ms/step
1/1 [==============================] - 0s 266ms/step
1/1 [==============================] - 0s 281ms/step
1/1 [==============================] - 0s 250ms/step
1/1 [==============================] - 0s 281ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 281ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 203ms/step
1/1 [==============================] - 0s 203ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 281ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 328ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 250ms/step
1/1 [==============================] - 0s 297ms/step
1/1 [==============================] - 0s 296ms/step
1/1 [==============================] - 0s 271ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 266ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 250ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 250ms/step
1/1 [==============================] - 0s 266ms/step
1/1 [==============================] - 0s 203ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 250ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 262ms/step
1/1 [==============================] - 0s 294ms/step
1/1 [==============================] - 0s 281ms/step
1/1 [==============================] - 0s 282ms/step
1/1 [==============================] - 0s 250ms/step
1/1 [==============================] - 0s 281ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 266ms/step
```

```
1/1 [==============================] - 0s 250ms/step
1/1 [==============================] - 0s 281ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 250ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 297ms/step
1/1 [==============================] - 0s 297ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 249ms/step
1/1 [==============================] - 0s 306ms/step
1/1 [==============================] - 0s 278ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 250ms/step
1/1 [==============================] - 0s 263ms/step
1/1 [==============================] - 0s 250ms/step
1/1 [==============================] - 0s 307ms/step
1/1 [==============================] - 0s 232ms/step
1/1 [==============================] - 0s 207ms/step
1/1 [==============================] - 0s 246ms/step
1/1 [==============================] - 0s 220ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 249ms/step
1/1 [==============================] - 0s 218ms/step
1/1 [==============================] - 0s 232ms/step
1/1 [==============================] - 0s 297ms/step
1/1 [==============================] - 0s 266ms/step
1/1 [==============================] - 0s 281ms/step
1/1 [==============================] - 0s 297ms/step
1/1 [==============================] - 0s 266ms/step
1/1 [==============================] - 0s 266ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 266ms/step
1/1 [==============================] - 0s 250ms/step
1/1 [==============================] - 0s 223ms/step
1/1 [==============================] - 0s 215ms/step
1/1 [==============================] - 0s 280ms/step
1/1 [==============================] - 0s 274ms/step
1/1 [==============================] - 0s 277ms/step
1/1 [==============================] - 0s 223ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 297ms/step
1/1 [==============================] - 0s 203ms/step
1/1 [==============================] - 0s 297ms/step
1/1 [==============================] - 0s 302ms/step
1/1 [==============================] - 0s 203ms/step
1/1 [==============================] - 0s 236ms/step
1/1 [==============================] - 0s 248ms/step
1/1 [==============================] - 0s 211ms/step
1/1 [==============================] - 0s 250ms/step
1/1 [==============================] - 0s 250ms/step
1/1 [==============================] - 0s 297ms/step
1/1 [==============================] - 0s 297ms/step
1/1 [==============================] - 0s 281ms/step
1/1 [==============================] - 0s 266ms/step
1/1 [==============================] - 0s 266ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 254ms/step
1/1 [==============================] - 0s 299ms/step
1/1 [==============================] - 0s 297ms/step
1/1 [==============================] - 0s 312ms/step
1/1 [==============================] - 0s 297ms/step
1/1 [==============================] - 0s 281ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 281ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 297ms/step
1/1 [==============================] - 0s 244ms/step
1/1 [==============================] - 0s 250ms/step
```

```
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 266ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 281ms/step
1/1 [==============================] - 0s 282ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 250ms/step
1/1 [==============================] - 0s 266ms/step
1/1 [==============================] - 0s 266ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 281ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 204ms/step
1/1 [==============================] - 0s 255ms/step
1/1 [==============================] - 0s 250ms/step
500 [D loss: 8.3005, acc.: 28.12%] [G loss: 5.8144]
WARNING:tensorflow:Compiled the loaded model, but the compiled metrics have yet to be bui
lt. `model.compile_metrics` will be empty until you train or evaluate the model.
```

```
WARNING:tensorflow:Compiled the loaded model, but the compiled metrics have yet to be bui
lt. `model.compile_metrics` will be empty until you train or evaluate the model.
WARNING:absl:Found untraced functions such as _jit_compiled_convolution_op, _jit_compiled
_convolution_op, _jit_compiled_convolution_op while saving (showing 3 of 3). These functi
ons will not be directly callable after loading.
```

```
INFO:tensorflow:Assets written to: E:/dl_miniproject/generator_500\assets
```

```
INFO:tensorflow:Assets written to: E:/dl_miniproject/generator_500\assets
```

```
Generator model saved at epoch 500
```

```
WARNING:absl:Found untraced functions such as _jit_compiled_convolution_op, _jit_compiled
_convolution_op, _update_step_xla while saving (showing 3 of 3). These functions will not
be directly callable after loading.
```

```
INFO:tensorflow:Assets written to: E:/dl_miniproject/discriminator_500\assets
```

```
INFO:tensorflow:Assets written to: E:/dl_miniproject/discriminator_500\assets
```

```
Discriminator model saved at epoch 500
```

```
WARNING:absl:Found untraced functions such as _update_step_xla, _jit_compiled_convolution
_op, _jit_compiled_convolution_op, _jit_compiled_convolution_op, _jit_compiled_convolutio
n_op while saving (showing 5 of 7). These functions will not be directly callable after l
oading.
```

```
INFO:tensorflow:Assets written to: E:/dl_miniproject/gan_500\assets
```

```
INFO:tensorflow:Assets written to: E:/dl_miniproject/gan_500\assets
```

```
GAN model saved at epoch 500
1/1 [==============================] - 0s 223ms/step
1/1 [==============================] - 0s 247ms/step
1/1 [==============================] - 0s 265ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 230ms/step
1/1 [==============================] - 0s 222ms/step
1/1 [==============================] - 0s 225ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 219ms/step
```

```
1/1 [==============================] - 0s 297ms/step
1/1 [==============================] - 0s 281ms/step
1/1 [==============================] - 0s 269ms/step
1/1 [==============================] - 0s 312ms/step
1/1 [==============================] - 0s 203ms/step
1/1 [==============================] - 0s 263ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 216ms/step
1/1 [==============================] - 0s 266ms/step
1/1 [==============================] - 0s 230ms/step
1/1 [==============================] - 0s 221ms/step
1/1 [==============================] - 0s 203ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 281ms/step
1/1 [==============================] - 0s 312ms/step
1/1 [==============================] - 0s 203ms/step
1/1 [==============================] - 0s 297ms/step
1/1 [==============================] - 0s 297ms/step
1/1 [==============================] - 0s 250ms/step
1/1 [==============================] - 0s 282ms/step
1/1 [==============================] - 0s 203ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 203ms/step
1/1 [==============================] - 0s 266ms/step
1/1 [==============================] - 0s 297ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 285ms/step
1/1 [==============================] - 0s 310ms/step
1/1 [==============================] - 0s 249ms/step
1/1 [==============================] - 0s 218ms/step
1/1 [==============================] - 0s 250ms/step
1/1 [==============================] - 0s 264ms/step
1/1 [==============================] - 0s 264ms/step
1/1 [==============================] - 0s 273ms/step
1/1 [==============================] - 0s 280ms/step
1/1 [==============================] - 0s 247ms/step
1/1 [==============================] - 0s 201ms/step
1/1 [==============================] - 0s 307ms/step
1/1 [==============================] - 0s 281ms/step
1/1 [==============================] - 0s 296ms/step
1/1 [==============================] - 0s 251ms/step
1/1 [==============================] - 0s 279ms/step
1/1 [==============================] - 0s 265ms/step
1/1 [==============================] - 0s 296ms/step
1/1 [==============================] - 0s 297ms/step
1/1 [==============================] - 0s 220ms/step
1/1 [==============================] - 0s 281ms/step
1/1 [==============================] - 0s 266ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 203ms/step
1/1 [==============================] - 0s 203ms/step
1/1 [==============================] - 0s 203ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 279ms/step
1/1 [==============================] - 0s 266ms/step
1/1 [==============================] - 0s 297ms/step
1/1 [==============================] - 0s 250ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 281ms/step
1/1 [==============================] - 0s 248ms/step
1/1 [==============================] - 0s 208ms/step
1/1 [==============================] - 0s 294ms/step
1/1 [==============================] - 0s 258ms/step
1/1 [==============================] - 0s 202ms/step
1/1 [==============================] - 0s 232ms/step
1/1 [==============================] - 0s 266ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 233ms/step
```

```
1/1 [==============================] - 0s 301ms/step
1/1 [==============================] - 0s 229ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 297ms/step
1/1 [==============================] - 0s 203ms/step
1/1 [==============================] - 0s 250ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 264ms/step
1/1 [==============================] - 0s 251ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 251ms/step
1/1 [==============================] - 0s 238ms/step
1/1 [==============================] - 0s 226ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 221ms/step
1/1 [==============================] - 0s 233ms/step
1/1 [==============================] - 0s 266ms/step
1/1 [==============================] - 0s 250ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 250ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 229ms/step
1/1 [==============================] - 0s 297ms/step
1/1 [==============================] - 0s 283ms/step
1/1 [==============================] - 0s 280ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 265ms/step
1/1 [==============================] - 0s 262ms/step
1/1 [==============================] - 0s 281ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 212ms/step
1/1 [==============================] - 0s 278ms/step
1/1 [==============================] - 0s 248ms/step
1/1 [==============================] - 0s 232ms/step
1/1 [==============================] - 0s 250ms/step
1/1 [==============================] - 0s 280ms/step
1/1 [==============================] - 0s 281ms/step
1/1 [==============================] - 0s 281ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 250ms/step
1/1 [==============================] - 0s 266ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 266ms/step
1/1 [==============================] - 0s 266ms/step
1/1 [==============================] - 0s 297ms/step
1/1 [==============================] - 0s 297ms/step
1/1 [==============================] - 0s 203ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 281ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 281ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 266ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 308ms/step
1/1 [==============================] - 0s 250ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 250ms/step
1/1 [==============================] - 0s 281ms/step
1/1 [==============================] - 0s 312ms/step
1/1 [==============================] - 0s 281ms/step
1/1 [==============================] - 0s 272ms/step
```

```
1/1 [==============================] - 0s 266ms/step
1/1 [==============================] - 0s 235ms/step
1/1 [==============================] - 0s 231ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 266ms/step
1/1 [==============================] - 0s 281ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 223ms/step
1/1 [==============================] - 0s 290ms/step
1/1 [==============================] - 0s 227ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 250ms/step
1/1 [==============================] - 0s 232ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 281ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 216ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 203ms/step
1/1 [==============================] - 0s 213ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 252ms/step
1/1 [==============================] - 0s 218ms/step
1/1 [==============================] - 0s 233ms/step
1/1 [==============================] - 0s 266ms/step
1/1 [==============================] - 0s 296ms/step
1/1 [==============================] - 0s 281ms/step
1/1 [==============================] - 0s 281ms/step
1/1 [==============================] - 0s 261ms/step
1/1 [==============================] - 0s 266ms/step
1/1 [==============================] - 0s 235ms/step
1/1 [==============================] - 0s 281ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 203ms/step
1/1 [==============================] - 0s 297ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 266ms/step
1/1 [==============================] - 0s 250ms/step
1/1 [==============================] - 0s 263ms/step
1/1 [==============================] - 0s 203ms/step
1/1 [==============================] - 0s 203ms/step
1/1 [==============================] - 0s 218ms/step
1/1 [==============================] - 0s 295ms/step
1/1 [==============================] - 0s 262ms/step
1/1 [==============================] - 0s 279ms/step
1/1 [==============================] - 0s 256ms/step
1/1 [==============================] - 0s 299ms/step
1/1 [==============================] - 0s 278ms/step
1/1 [==============================] - 0s 296ms/step
1/1 [==============================] - 0s 311ms/step
1/1 [==============================] - 0s 201ms/step
1/1 [==============================] - 0s 203ms/step
1/1 [==============================] - 0s 297ms/step
1/1 [==============================] - 0s 266ms/step
1/1 [==============================] - 0s 294ms/step
1/1 [==============================] - 0s 266ms/step
1/1 [==============================] - 0s 282ms/step
1/1 [==============================] - 0s 259ms/step
1/1 [==============================] - 0s 288ms/step
1/1 [==============================] - 0s 286ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 281ms/step
1/1 [==============================] - 0s 270ms/step
1/1 [==============================] - 0s 281ms/step
1/1 [==============================] - 0s 200ms/step
1/1 [==============================] - 0s 251ms/step
1/1 [==============================] - 0s 207ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 297ms/step
1/1 [==============================] - 0s 281ms/step
```

```
1/1 [==============================] - 0s 203ms/step
1/1 [==============================] - 0s 266ms/step
1/1 [==============================] - 0s 203ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 281ms/step
1/1 [==============================] - 0s 233ms/step
1/1 [==============================] - 0s 278ms/step
1/1 [==============================] - 0s 203ms/step
1/1 [==============================] - 0s 203ms/step
1/1 [==============================] - 0s 281ms/step
1/1 [==============================] - 0s 262ms/step
1/1 [==============================] - 0s 235ms/step
1/1 [==============================] - 0s 202ms/step
1/1 [==============================] - 0s 203ms/step
1/1 [==============================] - 0s 235ms/step
1/1 [==============================] - 0s 203ms/step
1/1 [==============================] - 0s 282ms/step
1/1 [==============================] - 0s 281ms/step
1/1 [==============================] - 0s 250ms/step
1/1 [==============================] - 0s 282ms/step
1/1 [==============================] - 0s 424ms/step
1/1 [==============================] - 0s 203ms/step
1/1 [==============================] - 0s 439ms/step
1/1 [==============================] - 0s 280ms/step
1/1 [==============================] - 0s 237ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 221ms/step
1/1 [==============================] - 0s 231ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 223ms/step
1/1 [==============================] - 0s 228ms/step
1/1 [==============================] - 0s 216ms/step
1/1 [==============================] - 0s 210ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 229ms/step
1/1 [==============================] - 0s 210ms/step
1/1 [==============================] - 0s 212ms/step
1/1 [==============================] - 0s 214ms/step
1/1 [==============================] - 0s 231ms/step
1/1 [==============================] - 0s 225ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 215ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 309ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 289ms/step
1/1 [==============================] - 0s 289ms/step
1/1 [==============================] - 0s 229ms/step
1/1 [==============================] - 0s 292ms/step
1/1 [==============================] - 0s 271ms/step
1/1 [==============================] - 0s 250ms/step
1/1 [==============================] - 0s 232ms/step
1/1 [==============================] - 0s 226ms/step
1/1 [==============================] - 0s 229ms/step
1/1 [==============================] - 0s 233ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 210ms/step
1/1 [==============================] - 0s 209ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 242ms/step
1/1 [==============================] - 0s 205ms/step
1/1 [==============================] - 0s 227ms/step
1/1 [==============================] - 0s 223ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 248ms/step
```

```
1/1 [==============================] - 0s 218ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 251ms/step
1/1 [==============================] - 0s 227ms/step
1/1 [==============================] - 0s 241ms/step
1/1 [==============================] - 0s 215ms/step
1/1 [==============================] - 0s 225ms/step
1/1 [==============================] - 0s 240ms/step
1/1 [==============================] - 0s 235ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 229ms/step
1/1 [==============================] - 0s 214ms/step
1/1 [==============================] - 0s 225ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 226ms/step
1/1 [==============================] - 0s 225ms/step
1/1 [==============================] - 0s 214ms/step
1/1 [==============================] - 0s 213ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 224ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 256ms/step
1/1 [==============================] - 0s 225ms/step
1/1 [==============================] - 0s 228ms/step
1/1 [==============================] - 0s 272ms/step
1/1 [==============================] - 0s 229ms/step
1/1 [==============================] - 0s 232ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 236ms/step
1/1 [==============================] - 0s 218ms/step
1/1 [==============================] - 0s 220ms/step
1/1 [==============================] - 0s 218ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 215ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 227ms/step
1/1 [==============================] - 0s 229ms/step
1/1 [==============================] - 0s 227ms/step
1/1 [==============================] - 0s 225ms/step
1/1 [==============================] - 0s 217ms/step
1/1 [==============================] - 0s 216ms/step
1/1 [==============================] - 0s 229ms/step
1/1 [==============================] - 0s 214ms/step
1/1 [==============================] - 0s 217ms/step
1/1 [==============================] - 0s 215ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 210ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 212ms/step
1/1 [==============================] - 0s 224ms/step
1/1 [==============================] - 0s 275ms/step
1/1 [==============================] - 0s 277ms/step
1/1 [==============================] - 0s 265ms/step
1/1 [==============================] - 0s 297ms/step
1/1 [==============================] - 0s 254ms/step
1/1 [==============================] - 0s 239ms/step
1/1 [==============================] - 0s 282ms/step
1/1 [==============================] - 0s 272ms/step
1/1 [==============================] - 0s 296ms/step
1/1 [==============================] - 0s 250ms/step
1/1 [==============================] - 0s 239ms/step
1/1 [==============================] - 0s 269ms/step
1/1 [==============================] - 0s 245ms/step
1/1 [==============================] - 0s 244ms/step
1/1 [==============================] - 0s 243ms/step
1/1 [==============================] - 0s 209ms/step
1/1 [==============================] - 0s 256ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 253ms/step
```

```
1/1 [==============================] - 0s 258ms/step
1/1 [==============================] - 0s 252ms/step
1/1 [==============================] - 0s 248ms/step
1/1 [==============================] - 0s 245ms/step
1/1 [==============================] - 0s 244ms/step
1/1 [==============================] - 0s 266ms/step
1/1 [==============================] - 0s 249ms/step
1/1 [==============================] - 0s 229ms/step
1/1 [==============================] - 0s 265ms/step
1/1 [==============================] - 0s 212ms/step
1/1 [==============================] - 0s 256ms/step
1/1 [==============================] - 0s 236ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 217ms/step
1/1 [==============================] - 0s 245ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 218ms/step
1/1 [==============================] - 0s 281ms/step
1/1 [==============================] - 0s 281ms/step
1/1 [==============================] - 0s 266ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 224ms/step
1/1 [==============================] - 0s 283ms/step
1/1 [==============================] - 0s 282ms/step
1/1 [==============================] - 0s 417ms/step
1/1 [==============================] - 0s 250ms/step
1/1 [==============================] - 0s 266ms/step
1/1 [==============================] - 0s 264ms/step
1/1 [==============================] - 0s 231ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 238ms/step
1/1 [==============================] - 0s 262ms/step
1/1 [==============================] - 0s 250ms/step
1/1 [==============================] - 0s 266ms/step
1/1 [==============================] - 0s 249ms/step
1/1 [==============================] - 0s 250ms/step
1/1 [==============================] - 0s 276ms/step
1/1 [==============================] - 0s 256ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 229ms/step
1/1 [==============================] - 0s 270ms/step
1/1 [==============================] - 0s 273ms/step
1/1 [==============================] - 0s 218ms/step
1/1 [==============================] - 0s 221ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 214ms/step
1/1 [==============================] - 0s 250ms/step
1/1 [==============================] - 0s 226ms/step
1/1 [==============================] - 0s 246ms/step
1/1 [==============================] - 0s 244ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 290ms/step
1/1 [==============================] - 0s 238ms/step
1/1 [==============================] - 0s 286ms/step
1/1 [==============================] - 0s 233ms/step
1/1 [==============================] - 0s 227ms/step
1/1 [==============================] - 0s 230ms/step
1/1 [==============================] - 0s 286ms/step
1/1 [==============================] - 0s 225ms/step
1/1 [==============================] - 0s 236ms/step
1/1 [==============================] - 0s 214ms/step
1/1 [==============================] - 0s 222ms/step
1/1 [==============================] - 0s 227ms/step
1/1 [==============================] - 0s 228ms/step
1/1 [==============================] - 0s 270ms/step
1/1 [==============================] - 0s 266ms/step
1/1 [==============================] - 0s 281ms/step
1/1 [==============================] - 0s 232ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 203ms/step
1/1 [==============================] - 0s 296ms/step
```

```
1/1 [==============================] - 0s 230ms/step
1/1 [==============================] - 0s 297ms/step
1/1 [==============================] - 0s 249ms/step
1/1 [==============================] - 0s 283ms/step
1/1 [==============================] - 0s 277ms/step
1/1 [==============================] - 0s 266ms/step
1/1 [==============================] - 0s 266ms/step
1/1 [==============================] - 0s 246ms/step
1/1 [==============================] - 0s 250ms/step
1/1 [==============================] - 0s 267ms/step
1/1 [==============================] - 0s 224ms/step
1/1 [==============================] - 0s 233ms/step
1/1 [==============================] - 0s 230ms/step
1/1 [==============================] - 0s 261ms/step
1/1 [==============================] - 0s 232ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 224ms/step
1/1 [==============================] - 0s 234ms/step
1/1 [==============================] - 0s 215ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 266ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 228ms/step
1/1 [==============================] - 0s 231ms/step
1/1 [==============================] - 0s 240ms/step
1/1 [==============================] - 0s 262ms/step
1/1 [==============================] - 0s 220ms/step
1/1 [==============================] - 0s 225ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 259ms/step
1/1 [==============================] - 0s 227ms/step
1/1 [==============================] - 0s 267ms/step
1/1 [==============================] - 0s 307ms/step
1/1 [==============================] - 0s 276ms/step
1/1 [==============================] - 0s 215ms/step
1/1 [==============================] - 0s 255ms/step
1/1 [==============================] - 0s 219ms/step
1/1 [==============================] - 0s 244ms/step
1/1 [==============================] - 0s 255ms/step
1/1 [==============================] - 0s 214ms/step
1/1 [==============================] - 0s 227ms/step
1/1 [==============================] - 0s 265ms/step
1/1 [==============================] - 0s 228ms/step
1/1 [==============================] - 0s 226ms/step
1/1 [==============================] - 0s 214ms/step
1/1 [==============================] - 0s 208ms/step
1/1 [==============================] - 0s 221ms/step
1/1 [==============================] - 0s 224ms/step
1/1 [==============================] - 0s 278ms/step
1/1 [==============================] - 0s 218ms/step
1/1 [==============================] - 0s 222ms/step
1000 [D loss: 1.3361, acc.: 62.50%] [G loss: 4.2074]
WARNING:tensorflow:Compiled the loaded model, but the compiled metrics have yet to be bui
lt. `model.compile_metrics` will be empty until you train or evaluate the model.
```

```
WARNING:tensorflow:Compiled the loaded model, but the compiled metrics have yet to be bui
lt. `model.compile_metrics` will be empty until you train or evaluate the model.
WARNING:absl:Found untraced functions such as _jit_compiled_convolution_op, _jit_compiled
_convolution_op, _jit_compiled_convolution_op while saving (showing 3 of 3). These functi
ons will not be directly callable after loading.
```

```
---------------------------------------------------------------------------
UnknownError                              Traceback (most recent call last)
Cell In[14], line 76
     73 print(f"{epoch} [D loss: {d_loss[0]:.4f}, acc.: {100 * d_loss[1]:.2f}%] [G loss:
{g_loss:.4f}]")
     75 # Save models at save_interval
---> 76 save_models(epoch, generator, discriminator, gan)

Cell In[14], line 11, in save_models(epoch, generator, discriminator, gan)
     10 def save_models(epoch, generator, discriminator, gan):
---> 11     generator.save(f'E:/dl_miniproject/generator_{epoch}', save_format='tf')  #
Using SavedModel format
```

```
using SavedModel format
     12     print(f"Generator model saved at epoch {epoch}")
     14     discriminator.save(f'E:/dl_miniproject/discriminator_{epoch}', save_format='t
f')  # Using SavedModel format

File D:\Documents\anacondaa\envs\tf_gpu_env\lib\site-packages\keras\utils\traceback_utils
.py:70, in filter_traceback.<locals>.error_handler(*args, **kwargs)
     67     filtered_tb = _process_traceback_frames(e.__traceback__)
     68     # To get the full stack trace, call:
     69     # `tf.debugging.disable_traceback_filtering()`
---> 70     raise e.with_traceback(filtered_tb) from None
     71 finally:
     72     del filtered_tb

File D:\Documents\anacondaa\envs\tf_gpu_env\lib\site-packages\tensorflow\python\eager\exe
cute.py:52, in quick_execute(op_name, num_outputs, inputs, attrs, ctx, name)
     50 try:
     51   ctx.ensure_initialized()
---> 52   tensors = pywrap_tfe.TFE_Py_Execute(ctx._handle, device_name, op_name,
     53                                       inputs, attrs, num_outputs)
     54 except core._NotOkStatusException as e:
     55   if name is not None:

UnknownError: {{function_node __wrapped__SaveV2_dtypes_21_device_/job:localhost/replica:0
/task:0/device:CPU:0}} Failed to WriteFile: \\?\E:\dl_miniproject\generator_1000\variable
s\variables_temp\part-00000-of-00001.data-00000-of-00001.tempstate9530439200577457704 : T
here is not enough space on the disk.
; operation in progress [Op:SaveV2]
```

image with cgan models

In [21]:

```
from rembg import remove
from PIL import Image
import numpy as np
import cv2

def remove_background_rembg(image_path, output_path):
    input_image = Image.open(image_path).convert("RGBA")
    output_image = remove(input_image)
    output_image = output_image.convert("RGB")
    output_image.save(output_path)

# Example usage:
human_image_path = r"E:\dl_miniproject\tejashvi.jpg"
output_path = r"E:\dl_miniproject\backgroundless_teja.jpg"
remove_background_rembg(human_image_path, output_path)
```

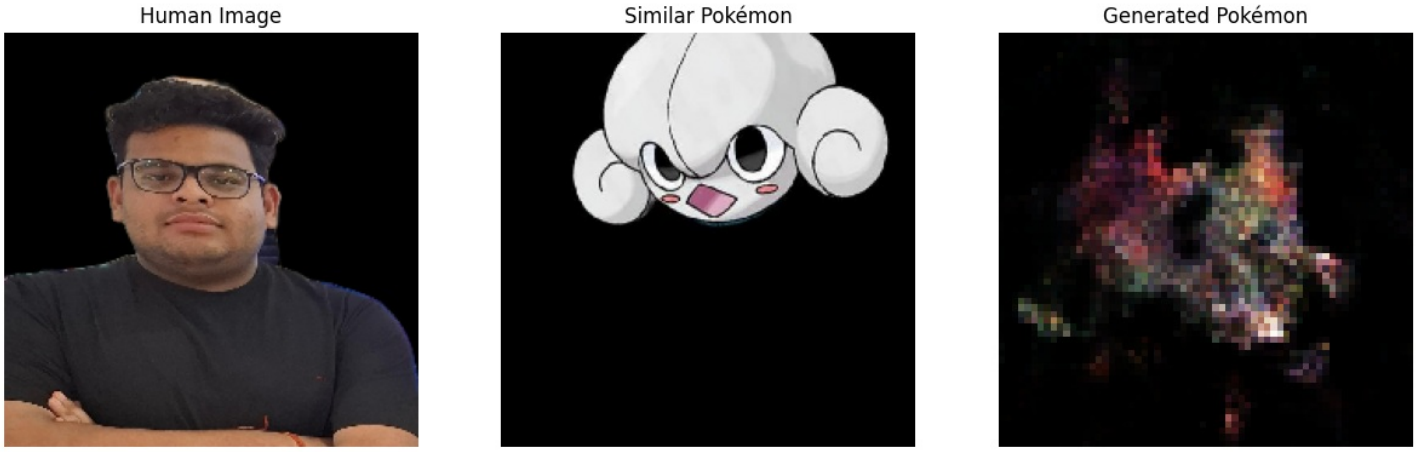
In [23]:

```
import numpy as np
from tensorflow.keras.applications.vgg16 import VGG16, preprocess_input
from tensorflow.keras.models import load_model
from tensorflow.keras.preprocessing import image
from sklearn.metrics.pairwise import cosine_similarity
import matplotlib.pyplot as plt
import os

# Step 1: Load the VGG16 model for feature extraction
vgg_model = VGG16(weights='imagenet', include_top=False, input_shape=(224, 224, 3))
vgg_model.trainable = False  # We don't need to train it

def extract_features(img_path):
    """Extract features from an image using VGG16."""
    img = image.load_img(img_path, target_size=(224, 224))
    img_array = image.img_to_array(img)
    img_array = np.expand_dims(img_array, axis=0)
    img_array = preprocess_input(img_array)
    features = vgg_model.predict(img_array)
    return features.flatten()
```

```python
# Step 2: Find the most similar Pokémon using cosine similarity
def find_most_similar_pokemon(human_features, pokemon_features, pokemon_filenames):
    similarities = cosine_similarity([human_features], pokemon_features)
    most_similar_idx = np.argmax(similarities)
    return pokemon_filenames[most_similar_idx], pokemon_features[most_similar_idx]

# Step 3: Load the generator model (SavedModel format)
generator_model_path = r"E:\dl_miniproject\rajuuu\generator_9999"
generator = load_model(generator_model_path)

def generate_personalized_image(generator, human_features, pokemon_features, latent_dim=1
00):
    # Sample random noise
    noise = np.random.normal(0, 1, (1, latent_dim))

    # Combine noise and human or Pokémon features
    condition_vector = human_features  # Alternatively, use pokemon_features based on you
r design

    # Concatenate noise and condition vector
    gen_input = np.concatenate([noise, condition_vector[np.newaxis, :]], axis=1)

    # Generate personalized Pokémon image
    generated_img = generator.predict(gen_input)

    # Rescale the image to [0, 1] range for displaying
    generated_img = 0.5 * generated_img + 0.5
    return generated_img

# Step 4: Load your Pokémon feature dataset (assumed precomputed and saved as npy files)
pokemon_features = np.load(r"E:\dl_miniproject\pokemon_features_3.npy")  # Shape: (4490,
25088)
pokemon_filenames = np.load(r"E:\dl_miniproject\pokemon_filenames_3.npy")  # Shape: (449
0,)

# Step 5: Display images
def display_images(human_image_path, pokemon_image_path, generated_image):
    """Display the human image, the most similar Pokémon, and the generated image."""

    # Load the human image
    human_img = image.load_img(human_image_path, target_size=(224, 224))
    human_img = np.array(human_img)

    # Load the most similar Pokémon image
    pokemon_img = image.load_img(pokemon_image_path, target_size=(224, 224))
    pokemon_img = np.array(pokemon_img)

    # Plot all three images: human, most similar Pokémon, and generated Pokémon
    fig, axes = plt.subplots(1, 3, figsize=(15, 5))

    # Display human image
    axes[0].imshow(human_img / 255.0)
    axes[0].set_title("Human Image")
    axes[0].axis('off')

    # Display Pokémon image
    axes[1].imshow(pokemon_img / 255.0)
    axes[1].set_title("Similar Pokémon")
    axes[1].axis('off')

    # Display generated image
    axes[2].imshow(generated_image[0])
    axes[2].set_title("Generated Pokémon")
    axes[2].axis('off')

    plt.show()

# Example usage
human_image_path = r"E:\dl_miniproject\backgroundless_teja.jpg"
human_features = extract_features(human_image_path)
```

```
# Find the most similar Pokémon
similar_pokemon_filename, similar_pokemon_features = find_most_similar_pokemon(human_feat
ures, pokemon_features, pokemon_filenames)
pokemon_image_path = os.path.join(r"E:\dl_miniproject\Pokemon_Dataset\pokemon_augmented",
similar_pokemon_filename)
print(f"Most similar Pokémon: {similar_pokemon_filename}")

# Generate personalized Pokémon image
personalized_image = generate_personalized_image(generator, human_features, similar_pokem
on_features)

# Display the human image, similar Pokémon, and generated Pokémon
display_images(human_image_path, pokemon_image_path, personalized_image)
```

```
WARNING:tensorflow:No training configuration found in save file, so the model was *not* c
ompiled. Compile it manually.
1/1 [==============================] - 0s 174ms/step
Most similar Pokémon: meditite_0_1842.jpg
1/1 [==============================] - 0s 240ms/step
```



In [10]:

```
"""from tensorflow.keras.models import load_model
import numpy as np
import matplotlib.pyplot as plt

# Define the paths to the saved models
generator_model_path = r"E:\dl_miniproject\generator_2000.h5"
discriminator_model_path = r"E:\dl_miniproject\discriminator_2000.h5"
gan_model_path = r"E:\dl_miniproject\gan_2000.h5"

# Load the saved models
generator = load_model(generator_model_path)
discriminator = load_model(discriminator_model_path)
gan = load_model(gan_model_path)

# Function to generate personalized Pokémon image
def generate_personalized_image(generator, human_features, latent_dim=100):
    # Sample random noise
    noise = np.random.normal(0, 1, (1, latent_dim))

    # Use the human features as the condition vector
    condition_vector = human_features  # Assuming human features are used for conditionin
g

    # Concatenate noise and human features
    gen_input = np.concatenate([noise, condition_vector[np.newaxis, :]], axis=1)

    # Generate personalized Pokémon image
    generated_img = generator.predict(gen_input)

    # Rescale the image to [0, 1] range for displaying
    generated_img = 0.5 * generated_img + 0.5
    return generated_img

# Example: Generate and display personalized Pokémon image
```

```
def display_generated_image(generator, human_features, latent_dim=100):
    # Generate the image
    personalized_image = generate_personalized_image(generator, human_features, latent_di
m)

    # Display the generated image
    plt.imshow(personalized_image[0])
    plt.axis('off')  # Hide axes
    plt.show()

human_image_path = r"E:\dl_miniproject\backgroundless_capyy.jpg"
human_features = extract_features(human_image_path)

# Display the generated personalized Pokémon image
display_generated_image(generator, human_features, latent_dim=100)
"""
```

Out[10]:

```
'from tensorflow.keras.models import load_model\nimport numpy as np\nimport matplotlib.py
plot as plt\n\n# Define the paths to the saved models\ngenerator_model_path = r"E:\\dl_mi
niproject\\generator_2000.h5"\ndiscriminator_model_path = r"E:\\dl_miniproject\\discrimin
ator_2000.h5"\ngan_model_path = r"E:\\dl_miniproject\\gan_2000.h5"\n\n# Load the saved mo
dels\ngenerator = load_model(generator_model_path)\ndiscriminator = load_model(discrimina
tor_model_path)\ngan = load_model(gan_model_path)\n\n# Function to generate personalized
Pokémon image\ndef generate_personalized_image(generator, human_features, latent_dim=100)
:\n    # Sample random noise\n    noise = np.random.normal(0, 1, (1, latent_dim))\n\n
# Use the human features as the condition vector\n    condition_vector = human_features
# Assuming human features are used for conditioning\n\n    # Concatenate noise and human
features\n    gen_input = np.concatenate([noise, condition_vector[np.newaxis, :]], axis=1
)\n\n    # Generate personalized Pokémon image\n    generated_img = generator.predict(gen
_input)\n\n    # Rescale the image to [0, 1] range for displaying\n    generated_img = 0.
5 * generated_img + 0.5\n    return generated_img\n\n# Example: Generate and display pers
onalized Pokémon image\ndef display_generated_image(generator, human_features, latent_dim
=100):\n    # Generate the image\n    personalized_image = generate_personalized_image(ge
nerator, human_features, latent_dim)\n\n    # Display the generated image\n    plt.imshow
(personalized_image[0])\n    plt.axis(\'off\')  # Hide axes\n    plt.show()\n\nhuman_imag
e_path = r"E:\\dl_miniproject\x08ackgroundless_capyy.jpg"\nhuman_features = extract_featu
res(human_image_path)\n\n# Display the generated personalized Pokémon image\ndisplay_gene
rated_image(generator, human_features, latent_dim=100)\n'
```

image with vae loss

In [18]:

```
from tensorflow.keras.models import load_model
from tensorflow.keras import layers
import tensorflow as tf
from tensorflow.keras.losses import mse
import numpy as np
from tensorflow.keras.preprocessing.image import load_img, img_to_array
import cv2

# Define your SamplingLayer to replace any lambda layer
class SamplingLayer(layers.Layer):
    def call(self, inputs):
        z_mean, z_log_var = inputs
        batch = tf.shape(z_mean)[0]
        dim = tf.shape(z_mean)[1]
        epsilon = tf.keras.backend.random_normal(shape=(batch, dim))
        return z_mean + tf.exp(0.5 * z_log_var) * epsilon

# Define your custom VAE loss function
def vae_loss(input_img, vae_output):
    reconstruction_loss = mse(input_img, vae_output)
    reconstruction_loss = tf.reduce_mean(reconstruction_loss, axis=[1, 2, 3])  # Adjust
dimensions if necessary
    reconstruction_loss *= 64 * 64 * 3  # assuming image dimensions are (64, 64, 3)
    kl_loss = 1 + z_log_var - tf.square(z_mean) - tf.exp(z_log_var)
    kl_loss = tf.reduce_mean(kl_loss, axis=-1)
    kl_loss *= -0.5
```

```
    return reconstruction_loss + kl_loss

# Load the encoder and decoder models separately
encoder = load_model(r"E:\dl_miniproject\rajuuu\vae_encoder_model.h5", custom_objects={'S
amplingLayer': SamplingLayer})
decoder = load_model(r"E:\dl_miniproject\rajuuu\vae_decoder_model.h5")

# Load the VAE model with custom objects
vae_model = load_model(r"E:\dl_miniproject\rajuuu\vae_model_1.h5", custom_objects={'Sampl
ingLayer': SamplingLayer, 'vae_loss': vae_loss})

# Load the generator, discriminator, and GAN models from their respective folders
generator = load_model(r"E:\dl_miniproject\rajuuu\generator_5000")
discriminator = load_model(r"E:\dl_miniproject\rajuuu\discriminator_5000")
gan = load_model(r"E:\dl_miniproject\rajuuu\gan_5000")

# Function to preprocess image
def preprocess_image(image_path, target_size=(64, 64)):
    image = load_img(image_path, target_size=target_size)
    image = img_to_array(image)
    image = image / 255.0  # Normalize to [0, 1]
    image = np.expand_dims(image, axis=0)  # Add batch dimension
    return image

# Function to get latent representation using encoder
def get_latent_representation(image, encoder_model):
    z_mean, z_log_var, z = encoder_model.predict(image)
    return z

from tensorflow.keras.layers import Dense

# Function to blend latent spaces (ensure dimensional consistency)
def blend_latent_spaces(human_latent, pokemon_latent, alpha=0.5):
    return alpha * human_latent + (1 - alpha) * pokemon_latent

# Function to generate an image from the latent space using the generator
def generate_image_from_latent(latent_vector, generator):
    # Transform the latent vector to the expected input shape for the generator
    expanded_latent = Dense(25188)(latent_vector)  # Adjust the size of the Dense layer
as needed
    generated_image = generator.predict(expanded_latent)
    return generated_image

# Optional sharpening function for the generated image
def sharpen_image(image):
    kernel = np.array([[0, -1, 0], [-1, 5, -1], [0, -1, 0]])  # Simple sharpening filter
    sharpened = cv2.filter2D(image, -1, kernel)
    return sharpened

# Example execution flow
human_image = preprocess_image(r"E:\dl_miniproject\backgroundless_kunal.jpg")
pokemon_image = preprocess_image(r"E:\dl_miniproject\Pokemon_Dataset\pokemon_augmented\am
aura_0_537.jpg")

# Example usage
human_latent = encoder.predict(human_image)[2]  # Ensure you are using the latent repres
entation
pokemon_latent = encoder.predict(pokemon_image)[2]  # Ensure you are using the latent re
presentation

# Blend latent spaces
blended_latent = blend_latent_spaces(human_latent, pokemon_latent, alpha=0.6)  # Adjust
alpha as needed

# Generate blended image with CGAN generator
generated_image = generate_image_from_latent(blended_latent, generator)

# Optional sharpening
final_output = sharpen_image(generated_image[0])  # Remove batch dimension before sharpe
ning

# Save or visualize the result
```

```
cv2.imwrite(r"E:\dl_miniproject\output_image.jpg", final_output * 255)  # Save image aft
er scaling to [0, 255]
```

```
WARNING:tensorflow:No training configuration found in the save file, so the model was *no
t* compiled. Compile it manually.
```

```
WARNING:tensorflow:No training configuration found in the save file, so the model was *no
t* compiled. Compile it manually.
```

```
WARNING:tensorflow:No training configuration found in the save file, so the model was *no
t* compiled. Compile it manually.
```

```
WARNING:tensorflow:No training configuration found in the save file, so the model was *no
t* compiled. Compile it manually.
```

```
WARNING:tensorflow:No training configuration found in save file, so the model was *not* c
ompiled. Compile it manually.
```

```
WARNING:tensorflow:No training configuration found in save file, so the model was *not* c
ompiled. Compile it manually.
```

```
1/1 [==============================] - 1s 513ms/step
1/1 [==============================] - 0s 39ms/step
1/1 [==============================] - 1s 1s/step
```

Out[18]:

```
True
```

In [ ]:

```

```